
EDA Bench: Testing Whether Language-Model Agents Build Functioning PCBs

Dhruv Saini
Independent Researcher
dhruv9saini@gmail.com

Abstract

Can a language-model agent reconstruct a PCB that behaves correctly at its connectors? We introduce EDA Bench, a benchmark with 35 KiCad tasks derived from public open-hardware projects. A system receives a prompt and a standard KiCad environment and must produce a complete project. The grader matches submitted connector pins to the pins named by the task, traces routed copper, applies deterministic test signals, runs ngspice where applicable, and checks KiCad design-rule output. It scores observable behavior at the board boundary rather than text overlap, screenshots, or component counts. Under the submitted grader, all 35 frozen human-designed references score 1.0 and all 35 severe structural-failure controls score at most 0.15. A larger revised-grader audit is more demanding: references average 0.957, swapped-pin faults average 0.177, and removed passives average 0.709, while benign representation changes reveal false rejections. Eight electrically identical PCB-layout controls, each made by rotating a complete reference layout by 180 degrees and translating it, all score 1.0 with no caps. These controls establish invariance to that rigid coordinate change, not to independently designed circuits or independently routed boards. Regrading all 210 archived harness outputs with the corrected open-connection rule preserves every build count and the leading archived configuration, while reducing its weighted score from 4.58% to 1.48%. A separate fresh medium-reasoning sweep builds all 35 projects and scores 2.18%. A predeclared six-task retained-trace audit spans all difficulty levels: under its separately frozen grader, all six projects build, their mean functional score is 4.17%, and none of the six agent workflows runs KiCad ERC or DRC before stopping. EDA Bench therefore exposes an inspectable gap between producing a parseable KiCad project and reproducing its required external electrical behavior, while making the grader’s remaining limits explicit.

1 Introduction

Language-model agents can edit files, run tools, inspect failures, and try again. Software benchmarks therefore judge the resulting program by running tests rather than comparing its text with a reference. SWE-bench, OSWorld, and ProgramBench are examples of this approach [Jimenez et al., 2023, Xie et al., 2024, Yang et al., 2026]. We apply the same basic idea to PCB reconstruction.

Hardware design has an analogous workflow. Engineers search for parts, inspect datasheets, draft schematics, route boards, run ERC and DRC checks, simulate where possible, and produce manufacturing files. For electronic design automation, however, execution-based testing is substantially harder. Correctness depends on routed connectivity, electrical behavior, manufacturability constraints, and external interfaces rather than only on executable program behavior. Existing CAD and EDA benchmarks cover important parts of this space, including text-to-CAD generation [Khan et al., 2024], geometric CAD validation [Epoch AI, 2026, CAD Arena, 2026, Barkley et al., 2026], schematic generation [Al Hasan et al., 2026, Zou et al., 2026, Luo et al., 2026], schematic understanding

[Lu et al., 2026], board-level schematic design [Qiu et al., 2026], hardware bug repair [Cui et al., 2026], and PCB placement and routing reasoning [Li et al., 2026]. These evaluations motivate execution-based assessment for engineering artifacts, but they do not directly test whether an agent can reconstruct a routed KiCad PCB that preserves the target board’s external electrical behavior.

EDA Bench evaluates complete agent systems on source-backed KiCad reconstruction tasks. A system receives a fixed prompt and runtime files, then writes a KiCad project to `/workspace/final_project`. KiCad provides open-source schematic, simulation, layout, rendering, and manufacturing tools [KiCad Project, 2026]. The tasks come from public hardware projects, and the main baselines can use the web. This makes the task auditable but also allows source discovery; we report that access policy as part of the evaluated system and do not claim closed-book circuit invention.

The scoring rule is easiest to state as an external-I/O test. For each task, a grader-side file lists the board pins visible to another device, the test signals to apply, the values to measure, and acceptable tolerances. The grader matches those pins to the submitted PCB, follows the realized copper, and runs deterministic checks or ngspice simulation [ngspice contributors, 2026]. A different internal circuit can score well if it produces the required boundary behavior. A visually plausible board can fail if a connector pin is isolated, swapped, shorted, or not driven by the needed circuitry.

Contributions. This paper makes five contributions.

- We introduce EDA Bench, a public benchmark of 35 source-backed KiCad PCB reconstruction tasks spanning connector breakouts, power-path circuits, controller boards, compute adapters, motor drivers, and high-density carrier boards.
- We define a reproducible harness protocol in which agents receive only task-visible runtime packs and must produce complete KiCad projects, while references, source snapshots, expected-I/O files, test cases, and grader internals remain in the grading pack.
- We develop an execution-based PCB grader that checks external pins, routed copper, ngspice behavior, task-family requirements, active circuitry, KiCad ERC/DRC diagnostics, and gross manufacturability.
- We validate the grader with frozen references, structural failures, realistic single-fault edits, leave-one-scorer-mechanism-out score recomputations, historical upstream versions, representation changes, and eight electrically identical rigid layout transforms, reporting both detected faults and grader blind spots.
- We report public-provenance baselines for six web-enabled harness configurations, regrade all 210 archived task outputs with the corrected open-connection rule, run one fresh complete medium-reasoning sweep, and audit unomitted retained root-agent event streams for a predeclared six-task sample spanning every difficulty. Current evaluated frontier-model harnesses remain far from functional PCB reconstruction.

2 Benchmark overview

EDA Bench evaluates PCB reconstruction as an execution task. For each task, a harness receives a fixed prompt, runtime task pack, tool environment, model configuration, and access policy, then must produce a complete KiCad project in `/workspace/final_project`. The grader parses the files, finds external connectors and routed copper, applies the task’s test signals and measurements, runs simulation where applicable, checks ERC and DRC diagnostics, and returns a normalized score.

Table 1 summarizes the pipeline. Each released task comes from a public hardware project and has a frozen reference design, but the evaluated runtime does not mount the reference, source snapshot, test cases, expected-I/O files, or grader implementation. This keeps the answer out of the container while leaving the released artifact inspectable. Because the sources are public and the main baselines are web-enabled, EDA Bench measures reconstruction under a declared public-source access policy rather than closed-book invention.

Scores should be interpreted as properties of complete evaluated harnesses, not isolated model weights. A harness may include prompting, web search, source discovery, KiCad tool use, repair loops, and project-generation logic. This systems-level framing is intentional because practical EDA

Table 1: EDA Bench evaluation pipeline. Each submission is evaluated as a complete harness output rather than as text, a screenshot, or a component list.

Stage	Artifact or operation
Task input	The harness receives a fixed prompt, runtime task pack, tool environment, model configuration, and access policy.
Harness execution	The agent operates in the Dockerized KiCad environment and writes a project to <code>/workspace/final_project</code> .
Project parsing	The grader locates KiCad files and extracts connectors, pads, nets, board outline, and routed copper geometry.
Port matching	Submitted connector pins are matched to the external pins and measurements listed by the task.
Execution checks	The grader applies KiCad ERC and DRC diagnostics, ngspice simulations, routed-I/O checks, parasitic models, and short-fault checks.
Score reporting	The grader returns a normalized task score, failure diagnostics, build status, and provenance artifacts.

Table 2: Task distribution and representative categories in EDA Bench.

Difficulty	Tasks	Representative task families
Very easy	2	Compact connector breakouts and small adapters
Easy	5	I/O breakouts, logic-level conversion, USB/UART basics
Medium	4	Power-path boards, serial transceivers, small controller boards
Hard	12	Compute adapters, sensor/camera boards, multi-connector carriers
Very hard	7	Dense mixed-signal boards and larger interface carriers
Extreme	5	Large baseboards and high-density compute boards

automation depends on whether an agent can operate the full workflow, not only on whether a model can emit plausible schematic or layout text in one shot.

The score concerns external electrical behavior rather than visual similarity. A board can use a different internal implementation if it exposes the required signals at the required pins. It can fail despite looking plausible if nets are missing, isolated, shorted, swapped, unrouted, or not driven by active circuitry. The score is not a fabrication certificate, regulatory review, thermal analysis, EMC validation, or vendor-accurate silicon simulation.

3 Benchmark tasks

EDA Bench contains 35 released source-backed tasks. Each task is anchored to a public upstream hardware project and a frozen reference design. The public task set covers six difficulty levels, shown in Table 2. Five additional candidate tasks were quarantined because their redistribution terms require permission or an acquisition-script replacement.

The task set was deliberately constructed to represent the major skills and project types in public, reproducible PCB reconstruction. Across six difficulty levels, it covers compact breakouts, power paths, controllers, camera and compute adapters, motor drivers, and dense carrier boards. A task is included only when it has (i) a redistributable public source or reproducible acquisition path, (ii) a complete KiCad board that can be frozen as a reference, (iii) identifiable external connectors, (iv) deterministic grader checks, and (v) a task family not already over-represented. This coverage-driven selection gives every task strong ground truth and avoids spending the evaluation budget on near-duplicates. Its external-validity boundary is public open-hardware designs with inspectable connector behavior: proprietary industry projects and analog-performance, RF, thermal, power-sequencing, and production-sign-off workflows are not yet covered.

The task catalog includes compact breakouts and converters such as `usb_c_female_breakout`, `mcp23017_breakout`, and `rs485_transceiver_breakout`. It also includes power and control boards such as `bq24295_power_path_board`, `picopd`, and `robotont_driver_board`, compute and camera or display adapters such as `cm4_csi_adapter`, `m2_pcie_adapter`, and `ov9281_dual_camera_board`, and large carrier boards such as `cm4_baseboard`, `jetson_orin_baseboard`, and `zynq_som`.

Each task has two artifact forms. The runtime task pack is mounted for the evaluated agent and contains the prompt, task metadata, and runtime assets. It does not include the reference project, source snapshot, test cases, expected-I/O files, or grader implementation. The full grading pack is public and adds the reference, upstream snapshot, controlled-fault submissions, license notes, `io_contract.json`, `functional_contract.json`, and local tests. The filenames remain for compatibility, but in plain language these two JSON files say which external pins exist, which signals to apply, what to measure, and which task-specific checks to run.

4 Harness protocol

A harness is evaluated as a complete system. It receives the task prompt and must produce a KiCad project in `/workspace/final_project`. Built-in harnesses run inside Docker with KiCad, ngspice, Python, Node, the declared web policy, and the configured model interface. The evaluated systems are Codex CLI with GPT-5.5 web search at four reasoning settings, plus Pi Coding Agent—an open-source terminal agent that can edit files and run shell tools [Zechner and contributors, 2026]—with Gemini 3.1 Pro Preview or DeepSeek V4 Pro.

Only the four Codex rows form a within-model reasoning-setting sweep. Gemini and DeepSeek were each run through Pi at one registered setting, `-thinking high`; no non-high Pi results exist. The two Pi rows use identical prompts, tools, web extension, image, and timeout, but different models and provider routes. Their high labels are not a calibrated common cost, so these rows do not measure reasoning-cost scaling.

The task body is byte-identical across all six configurations, and the full prompt is fixed within the Codex sweep and between the Pi models. Codex alone adds wrapper instructions and exposes different tools. Prompt text therefore cannot explain either within-runner comparison, but no causal prompt ablation or isolated Codex–Pi comparison was run.

The protocol evaluates complete systems, not only model weights. Baseline reports retain prompts, commands, usage, grader outputs, and projects. Raw transcripts were omitted in the original campaigns; the separate local trace study below required their retention.

For the separate local retained-trace study, transcript retention was required rather than optional and external upload was disabled. The frozen protocol selected the first task in manifest order at each of the six difficulty levels before any results were observed, ran one fresh GPT-5.5 web-medium session per task, and retained the complete structured root-agent event stream. This campaign uses the pre-Harbor runner and a separately pinned task-pack revision, so its scores are not merged into Table 6. Our workflow analysis uses command, file-change, web, collaboration, and final-artifact events; it does not use hidden reasoning text. One task records a subagent spawn but does not retain that child’s stream, so all event and action counts are explicitly root-agent counts.

5 Scoring method

The grader asks a concrete question: if another device were connected to the named board pins, would the submitted PCB expose the required paths and responses? Missing or malformed projects receive zero. For a parseable project, the grader runs seven steps: locate the KiCad files; run ERC and DRC; extract connectors, pads, nets, and routed copper; match submitted pins to the task’s external pins; apply the listed test signals and measurements; compute family-specific checks and failure caps; and return a score with diagnostics.

The primary score is a normalized task score in $[0, 1]$. A benchmark aggregate is a per-task difficulty-weighted mean:

$$S = \frac{\sum_{i=1}^N w(d_i) s_i}{\sum_{i=1}^N w(d_i)}, \quad w(d_i) \in \{1.0, 1.5, 2.0, 3.0, 4.0, 5.0\}. \quad (1)$$

We also report the unweighted mean over tasks. Build success means that the submission produced a KiCad project that the grader could parse and evaluate. It is not equivalent to functional success.

The main score comes from three observable properties: whether connector pins mean the right thing, whether routed copper realizes the required paths, and whether the needed active circuitry and

Table 3: Representative task-family checks layered on the shared external-I/O grader. These are deterministic benchmark checks, not vendor-accurate silicon or channel models.

Task family	Declared behavior	Common hard caps
Connector breakouts and simple adapters	Connector pins and power/ground pads, continuity, shorts, routed coverage, pin/net semantic agreement	Swapped pins, split nets, isolated pads, power-ground shorts
Serial and level-shifting interfaces	USB/UART/RS-485/logic-side boundary ports, signal transfer families, active-device realization	Missing active path, weak external mapping, missing routed I/O
Power-path and charger boards	Input, battery/load, configuration, and status ports, power path presence, external safety	Missing active power role, shorts, missing component-function profile
Camera/display/compute carriers	High-density connector families and differential pairs, routed families, pair plausibility	Missing high-speed pair families, weak routing, shorts, split nets
Large mixed-signal baseboards	Multi-rail power, GPIO, debug, serial, high-speed, and peripheral interfaces	Incomplete connector semantics, missing active realization, shorts

gross physical layout are plausible. Deterministic ngspice signals are applied at declared external pins where the task supports simulation. Missing copper, swapped pins, shorts, absent or unpowered active devices, missing high-speed pair families, and gross fabrication problems cap the score. Exact reference designators, component inventory, footprint similarity, outline similarity, and route-shape similarity are diagnostics rather than primary score signals.

The measurements produce partial scores for pin mapping, routed continuity, simulated response, and task-family checks. Severe structural failures then cap the combined score. This allows partial credit for partially correct behavior while limiting credit for a board that passes unrelated checks but has a short, an isolated connector, swapped pins, or no active circuit.

6 Artifact validation

Because EDA Bench uses task-specific checks, the release includes controlled boards that calibrate the score. The positive control is the frozen reference. The severe negative control removes active internal circuitry when available; for passive boards it removes an external boundary. The submitted release required the positive control to score 1.0 and the severe negative control to score below 0.75. We retain those original checks and separately report the complete revised-grader audit rather than assuming that the correction preserves them.

What validation establishes. The grader is not a vendor-accurate electrical simulator. Its intended validity criterion is narrower: score changes should track changes to the external behavior named by the task. Positive controls test that real human-designed boards score well. Fault controls test changes that break routing, power, pin mapping, or active circuitry. Representation controls test whether benign edits change the score; as Table 4 shows, several do.

A full local sweep on May 6, 2026 verified the submitted grader on all 35 released task canaries: every pass canary scored 1.0, and every fail canary scored at most 0.15. The generated-mutation sweep graded 162 mutation canaries across the original 40-task candidate set. Mutations covered missing board outlines, unrouted external nets, isolated external pads, power or signal pads tied to ground, and missing high-speed-like routes for high-speed carrier tasks.

The revised grader adds a dedicated zero cap when a submission has more KiCad-reported open connections than its task reference. On the complete 35-task fault corpus, unmodified references average 0.957: 33 score at least 0.95 and two score 0.25. The latter two are false rejections introduced by stricter physical checks. Internal-reference-designator variants average 0.855, so the full audit

Table 4: Grader-validation evidence. These controls calibrate a boundary-behavior score, not complete manufacturing or vendor-silicon review.

Evidence	What it supports
Submitted grader: 35/35 references score 1.0	The released tasks are solvable, and the submitted grader accepts the selected upstream designs.
Submitted grader: 35/35 structural fail canaries score at most 0.15	The submitted score is not reducible to file existence, KiCad syntax, or component-shell matching.
162 candidate-set mutation controls pass validation	The grader is sensitive to shorts, unrouted nets, isolated pads, missing outlines, and missing high-speed-like routing.
Revised grader: references average 0.957	33/35 references score at least 0.95; two false rejections score 0.25.
Revised grader: internal-refdes variants average 0.855	Internal names are not fully invariant under the revised grader.
Rigidly transformed PCB layouts score 1.0	All eight electrically identical 180-degree-rotated and translated layouts score 1.0 with no caps.
Renaming the two USB-C connector references scores 0.992	External connector names have a small, unintended effect.
Changing plausible USB-C route widths to 0.35 mm scores 0.25	The current grader is too sensitive to a benign routing representation change.
Replacing six USB-C signal-net names with generic names scores 0.65	The current grader partly relies on semantic net names even when topology is unchanged.
Tiny-trace USB-C control scores at most 0.35	Manufacturability caps affect scores for physically implausible routing.

also rejects the earlier general claim that internal naming is always harmless. The smaller USB-C representation controls give exact examples: renaming two connector references scores 0.992, changing every applicable route to a plausible 0.35 mm width scores 0.25, and replacing six signal-net names with generic names scores 0.65 despite unchanged topology.

These controls do not show that the grader replaces expert board review or captures vendor-specific silicon behavior. They show that it accepts most selected reference boards and rejects several common function-breaking edits. They also expose representation sensitivity that the original validation discussion understated: reference designators, signal-net names, and even a plausible uniform route-width change can reduce the score. This is a grader limitation, not an electrical failure of those boards.

Realistic single-fault sweep. We applied ten additional deterministic edits across all applicable references. Table 5 reports every fault family under the revised grader rather than only the successful controls. Across 295 applicable fault rows, 271 (91.9%) lower the final score and 24 do not. A high final score and a completely unchanged score are different outcomes: 75 fault rows end at or above 0.95, but 58 of those still lose score and only 17 are unchanged. In particular, all nine high-scoring single-pad disconnects and all 23 high-scoring passive removals lose score; every pin swap and signal-to-ground edit also loses score.

Across all score levels, the 24 unchanged rows consist of 19 genuine false negatives, four edits outside the public engineering brief, one electrically redundant edit, and no no-ops under a manual requirement-level audit. Seven start from references already capped at 0.25, so changed subscores can be hidden by the pre-existing DRC cap. Table 13 gives the complete score-response counts. The grader detects important failures, but it is not a certificate that every component or route is correct.

Oracle-component ablation. We also remove one scorer mechanism at a time from the archived revised-grader diagnostics, renormalize the remaining weighted subscores, and remove only that mechanism’s caps. Every ablation leaves positive mean reference–fault separation. Even without the most influential KiCad-rule mechanism, the paired gap remains 0.198 and the 99 archived saved projects average only 0.045. Removing that mechanism reduces separation by 0.402; removing internal-circuit, connectivity, interface, or high-speed checks reduces it by 0.018, 0.017, 0.015, and 0.004. Simulation and fabrication checks have essentially zero marginal effect in this corpus. Table 14 reports the full counterfactual recomputation, which is not a rerun of KiCad or ngspice and is not additive because several mechanisms can constrain the same score through the final cap.

Table 5: Single-fault sweep over the 35 frozen references. “Detected” counts scores at or below 0.75; applicability differs when a board lacks the edited feature.

Board variant	Applicable	Mean score	Detected
Renamed internal reference designators	35	0.855	11
Two external connector pins swapped	35	0.177	32
One external pad disconnected	35	0.459	26
One external route removed	34	0.235	27
One signal tied to ground	35	0.221	30
One active power pad disconnected	31	0.456	23
Active device loses all power or ground	31	0.398	26
One passive component removed	34	0.709	11
One active device removed	32	0.293	23
One high-speed route removed	28	0.238	22

Table 6: Complete web-enabled harness baselines. “Submitted” is the paper’s weighted score. The revised columns regrade the same saved projects after the parser and excess-open correction.

Harness	Submitted	Revised	Revised unweighted	Builds
Codex GPT-5.5 web low	0.0124	0.0014	0.0014	31/35
Codex GPT-5.5 web medium	0.0279	0.0092	0.0129	35/35
Codex GPT-5.5 web high	0.0293	0.0014	0.0014	11/35
Codex GPT-5.5 web xhigh	0.0458	0.0148	0.0186	21/35
Pi Gemini 3.1 Pro Preview web high	0.0113	0.0000	0.0000	1/35
Pi DeepSeek V4 Pro web high	0.0000	0.0000	0.0000	0/35

Electrically identical rigid layout transforms. We tested whether the grader changes its score when a working PCB is moved to different absolute coordinates. For eight tasks spanning very easy through hard, we preserved the complete reference schematic, components, connectivity, relative placement, and routing, then rotated every PCB item by 180 degrees about the board center and translated the complete layout by 12.7 mm in X and 7.62 mm in Y. Every saved PCB is byte-distinct from its reference. In the USB-C breakout and logic-level-converter controls, we omitted the non-electrical FD1 fiducial because KiCad 10 reserialization otherwise turns its legacy custom-pad geometry into a new self-overlap DRC error.

All eight transformed projects build and score exactly 1.0 with no score caps. This establishes invariance to the tested rigid rotation and translation. The controls are electrically identical, reference-derived layouts; they are not independently designed circuits or independently routed PCBs. Earlier prompt-authored boards are excluded from valid-alternative evidence because a contract-level audit found wrong connector mappings or missing task-required functional blocks.

7 Experiments

We evaluate six web-enabled harness configurations on the 35 released tasks. All runs use the same task prompts, task packs, Docker image, and grading protocol. Table 6 reports both the paper’s May 6, 2026 scores and a no-model regrade of the exact archived final projects under the revised grader.

No evaluated harness achieves a high absolute score. Under the corrected grader, the medium reasoning-level Codex harness builds all 35 released projects but scores 0.92%, while xhigh builds 21 and scores 1.48%. Exactly 38 of 210 task scores decrease, none increases, and all 210 build outcomes remain unchanged. Every decrease is attached to the new excess-open cap. The revised order is xhigh above medium, low tied with high, and both Pi configurations at zero. The central result is stronger after the correction: producing parseable KiCad output is a poor proxy for reproducing the required external behavior.

The regrade covers all 210 task/configuration rows and verifies all 341 archived project files against the pinned provenance revision. Submitted and revised port, selected-track, via, total-track, and total-via counts are identical in every row, so the score changes are not caused by geometry drift or different saved outputs.

One fresh complete sweep. We also ran one new 35-task Codex GPT-5.5 web-medium sweep with one fresh model session per task. It scores 0.02183 under the difficulty weighting and 0.02286 under an unweighted task mean. The saved Zynq project originally exceeded the grader’s 90-second

ngspice timeout; reggrading that exact four-file project with a 600-second timeout completes in 477 seconds, changes build success from 34/35 to 35/35, and leaves its task score at zero. This is useful confirmation that a second build-all run still has low functional performance. One new sweep is not enough to estimate run-to-run variance or make a statistical ranking claim.

Predeclared retained-trace audit. Before observing results, we froze a separate local pre-Harbor study selecting the first task in manifest order at each difficulty. Fresh GPT-5.5 web-medium sessions build 6/6 projects but score 0.10, 0.05, 0, 0.10, 0, and 0 under that study’s frozen grader (mean 0.04167). The unomitted root streams contain 872 events, 226 completed commands, 36 file changes, and 15 web operations. None of the six agents invokes KiCad ERC or DRC before stopping, although they do run custom checks and repair files. Because `kicad-cli` is unavailable in this frozen agent image, this is both a workflow failure and a harness affordance to fix. Appendix A gives the task-level results, exact provenance, and root-trace limitation.

Statistical interpretation. The submitted result table reports one saved project per task and configuration, not repeated independent model runs. We therefore do not claim statistically significant pairwise model rankings. As a task-composition sensitivity check on those submitted scores, we resample the 35 tasks with replacement. The xhigh row has a task-bootstrap 95% interval of [0.0282, 0.0662] and ranks first in 79.1% of resamples; the medium and high rows have intervals [0.0169, 0.0411] and [0.0085, 0.0587]. The same xhigh row remains first under the paper’s difficulty weights (0.0458), an unweighted task mean (0.0500), and equal weighting of the six tiers (0.0475). These intervals measure sensitivity to which tasks are included; they are not stochastic-run confidence intervals. The revised-grader comparison instead gives a deterministic matched regrade of the same projects.

7.1 Access-policy calibration

EDA Bench records web access as part of the system definition, but the archived no-web attempts do not support a web-versus-no-web comparison. Manual provenance review found that all six no-web campaigns used invalidated authentication tokens and failed before producing usable model outputs. They are infrastructure failures and are removed from the capability analysis. A valid access-policy ablation would require fresh authenticated repeats with identical model, prompt, runtime, and task pack. We do not have that experiment.

Public-source reuse audit. Because the main harnesses can browse public upstream repositories, we compared every parseable saved board with its frozen reference. Across the four Codex configurations there are zero byte-identical PCB files and zero matches after canonicalizing reference-designator names and ordering. Median topology similarity ranges from 0.216 to 0.520 and median route-geometry similarity is 0.000. The only near-reference route case is the sole parseable Pi/Gemini board (route Jaccard 0.825, topology-signature Jaccard 1.000).

A coordinate audit explains why that similarity is not functionality: the conversion translates footprint-local pad coordinates that should remain local, leaving 70 of 147 external pads isolated and seven excess split nets. This sole nonzero Gemini task therefore receives 0.30 under the submitted grader (0.8571% unweighted overall) and zero under the corrected open-connection rule. Route Jaccard diagnoses wholesale final-artifact reuse, not function or all source assistance.

The archived corpus cannot support a step-by-step cross-model trace analysis: all 99 saved Table 6 projects have explicit omission records accounting for 16,486 parsed events. Final artifacts remain, but they do not recover tool order or browsing history. Two earlier preliminary USB-C runs retain full Codex-low and Pi/Gemini-high traces; Appendix B reports that one-task contrast separately and does not generalize it to Table 6.

The separate six-task trace confirms the distinction: ASAC uses an exact upstream XML netlist, ADAU1452 downloads exact upstream KiCad files before emitting a byte-distinct derivative, and CM4 uses public documentation without an observed upstream-project copy. These observations apply only to the six traced runs; Appendix A gives the full audit.

8 Failure analysis

The leading archived row scores 1.48% under the revised grader, and the fresh full medium sweep scores 2.18%. Two medium sweeps build every task and still fail most functional checks, showing that parseability is not functional reconstruction. The setting rows are complete-harness outcomes rather than a clean model-scaling curve: the archive contains one output per task, no Pi setting sweep, and several authentication or provider-stream failures.

We audited the 99 saved projects using one binary indicator per project and strict submitted-grader mechanism. Categories overlap. Wrong external-pin mapping occurs in 76/99 projects, isolated or split connectivity in 73/99, shorts in 70/99, missing required circuits in 70/99, unpowered active devices in 18/99, and high-speed-routing failures in 12/99. Appendix B reports counts by harness and broader incomplete cases.

All six retained-trace agents repair files and build, but none runs KiCad ERC, DRC, or ngspice before stopping. Parser checks and exact upstream information do not prevent wrong connector meaning, incomplete active paths, or shorts.

The evidence motivates a typed connector-to-net plan, an explicit required-block and active-power checklist, formal ERC/DRC and electrical checks, and diagnostics-driven repair before termination. The grader responds to 271/295 controlled faults, but 19 genuine unchanged faults prevent treating it as the sole oracle. This ordering is observational; Appendix B gives the proposed paired ablation.

9 Related work

Execution-based agent benchmarks run produced artifacts or actions [Jimenez et al., 2023, Xie et al., 2024, Yang et al., 2026]; CAD and EDA benchmarks cover geometry, schematics, hardware repair, and PCB reasoning [Khan et al., 2024, Barkley et al., 2026, Qiu et al., 2026, Cui et al., 2026, Li et al., 2026]. EDA Bench instead requires a complete routed KiCad project and scores realized external-I/O behavior and copper geometry. Appendix C gives the full comparison.

10 Limitations

EDA Bench measures public-source reconstruction under declared prompts, tools, provenance, and web policy, not closed-book invention. Its 35 tasks span board families, functional skills, and six levels while keeping complete tool-using sweeps practical; other expensive open-ended benchmarks use 7–20 tasks [Huang et al., 2024, Wijk et al., 2024, Starace et al., 2025]. Proprietary designs and several analog, RF, thermal, and sign-off workflows remain outside this evidence, and license constraints quarantine five candidate tasks.

The grader is not a manufacturing certificate and does not prove safety, regulatory or protocol compliance, or production readiness. It has measured blind spots: 19 controlled faults leave scores unchanged, 58 detected faults still score at least 0.95, and some behavior-preserving representation changes are rejected. Rigid controls establish invariance only to the tested coordinate transform; we have not measured agreement with blinded PCB engineers.

One fresh sweep and task bootstraps do not estimate run variance. There is no non-high Pi sweep or prompt ablation, and high is not a shared cost scale. The six-task trace is a pre-Harbor Codex case study; the 99 reported traces and one spawned child stream are absent. Appendix D gives details.

11 Conclusion

EDA Bench shows that parseable agent-built PCBs often fail external-I/O checks. Regrading, a fresh sweep, and a workflow audit locate these failures, while validation controls delimit remaining equivalence coverage.

References

Khandakar Shakib Al Hasan, Syed Rifat Raiyan, Hasin Mahtab Alvee, and Wahid Sadik. CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural

- Language Prompts. arXiv:2601.04505, 2026. URL: <https://arxiv.org/abs/2601.04505>.
- Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. CADSmith: Multi-Agent CAD Generation with Programmatic Geometric Validation. arXiv:2603.26512, 2026. URL: <https://arxiv.org/abs/2603.26512>.
- CAD Arena. CAD Arena: Open Benchmark for AI-Generated Parametric CAD. 2026. URL: <https://cadarena.dev/>.
- Fan Cui, Hongyuan Hou, Zizhang Luo, Chenyun Yin, and Yun Liang. HWE-Bench: Benchmarking LLM Agents on Real-World Hardware Bug Repair Tasks. arXiv:2604.14709, 2026. URL: <https://arxiv.org/abs/2604.14709>.
- Epoch AI. CadEval. 2026. URL: <https://epoch.ai/benchmarks/cad-eval>.
- Marius Hobbhahn. SWE-bench Verified Mini. GitHub repository, 2025. URL: <https://github.com/mariushobbhahn/SWEBench-verified-mini>.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating Language Agents on Machine Learning Experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL: <https://arxiv.org/abs/2310.03302>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. URL: <https://arxiv.org/abs/2310.06770>.
- Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL: <https://arxiv.org/abs/2409.17106>.
- KiCad Project. KiCad Documentation. 2026. URL: <https://docs.kicad.org/>.
- Jindong Li, Lianrong Chen, Bin Yang, Jiadong Zhu, Ying Wang, Yuzhe Ma, and Menglin Yang. PCB-Bench: Benchmarking LLMs for Printed Circuit Board Placement and Routing. In *International Conference on Learning Representations*, 2026. URL: <https://openreview.net/forum?id=Q5QLu7XTWx>.
- Taiting Lu, Kaiyuan Lin, Yuxin Tian, Yubo Wang, Muchuan Wang, Sharique Khatri, Akshit Kartik, Yixi Wang, Amey Santosh Rane, Yida Wang, Yifan Yang, Yi-Chao Chen, Yincheng Jin, and Mahanth Gowda. OmniSch: A Multimodal PCB Schematic Benchmark For Structured Diagram Visual Reasoning. arXiv:2604.00270, 2026. URL: <https://arxiv.org/abs/2604.00270>.
- Qinpei Luo, Ruichun Ma, Xinyu Zhang, and Lili Qiu. SchGen: PCB Schematic Generation with Semantic-Grounded Code Representations. OpenReview preprint, 2026. URL: <https://openreview.net/forum?id=TyWs6rWWHb>.
- ngspice contributors. ngspice, the open-source SPICE circuit simulator. 2026. URL: <https://ngspice.sourceforge.io/>.
- Weibo Qiu, Yinhao Xiao, and Runyu Pan. HWE-Bench: Can Language Models Perform Board-level Schematic Designs? arXiv:2603.18102, 2026. URL: <https://arxiv.org/abs/2603.18102>.
- Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL: <https://arxiv.org/abs/2402.14992>.
- Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. PostTrainBench: Can LLM Agents Automate LLM Post-Training? arXiv:2603.08640, 2026. URL: <https://arxiv.org/abs/2603.08640>.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI’s Ability to Replicate AI Research. arXiv:2504.01848, 2025. URL: <https://arxiv.org/abs/2504.01848>.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, Elena Ericeva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. arXiv:2411.15114, 2024. URL: <https://arxiv.org/abs/2411.15114>.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. URL: <https://arxiv.org/abs/2404.07972>.

Mario Zechner and contributors. Pi Coding Agent. Open-source terminal coding-agent harness, 2026. URL: <https://github.com/badlogic/pi-mono/tree/main/packages/coding-agent>.

John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. ProgramBench: Can Language Models Rebuild Programs From Scratch? arXiv:2605.03546, 2026. URL: <https://arxiv.org/abs/2605.03546>.

Huanghao Zou, Peng Han, Emad Nazerian, and Alex Q. Huang. PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards. arXiv:2602.00510, 2026. URL: <https://arxiv.org/abs/2602.00510>.

A Artifact availability and provenance

The source repository contains the runner and harness code, task-pack loader and grader support, public documentation, package tests, and the task-pack manifest. Python wheels and source distributions exclude concrete task instances, reference projects, upstream snapshots, and canary submissions. The Hugging Face task dataset contains runtime task packs, full grader task packs, prompts, references, upstream snapshots, canary submissions, task metadata, Croissant metadata, and license/provenance notes.

The current public task dataset is hosted at <https://huggingface.co/datasets/eda-bench-neurips-2026/eda-bench-tasks>. The paper results use task-pack revision c64b9169d6d8c0d73614e14c7050cf036aeb1559. The public provenance dataset is hosted at <https://huggingface.co/datasets/eda-bench-neurips-2026/eda-bench-provenance>. The public code release is hosted at <https://github.com/CAD-bench/eda-bench>.

Run provenance includes task prompts, command lines, usage summaries, grader outputs, final projects, and generated artifacts. In the archived Table 6 corpus, all 99 saved projects retain final responses, session records, command records, and final projects, but all 99 model/tool event files are explicit omission records because transcript upload was disabled. They account for 16,486 omitted parsed events, 23,283,836 omitted stdout bytes, and 17,338 omitted stderr bytes. This supports artifact-level auditing but not reconstruction of the agents’ step-by-step actions.

The separate retained-trace study used the predeclared protocol eda-agent-trace-study-v2-20260728. It pinned task-pack revision d4dfbcc6354c31c426af557d69e35bcedade14d2, submitted-code base fadf56222d3880e35d06988eab9ae5b599442d2, agent-image digest 98083f6e67873573dfddd35a9a16e0d5afb2c52a606dd470e17a39d6fdc18950, and clean trace-only execution commit 96eab64fdb25234a09bfbf5c2cd6019cb6937ca1. Its six retained root streams contain 872 events, including 226 completed commands, 36 completed file changes, 15 web operations, and one subagent spawn, with zero omitted transcript bytes. Every command, file-change, and web start has a matching completion. We analyze observable actions, final messages, final artifacts, and grader evidence rather than hidden reasoning text. The ASAC run records a subagent spawn but does not retain the child stream, so these counts cover root-agent activity only. This pre-Harbor study is separate from the current public Harbor protocol and from the task revision used in Table 6.

Table 7: Predeclared retained-trace sample. Scores use the study’s separately frozen grader. ERC/DRC indicates whether the agent invoked formal KiCad verification before stopping.

Task	Difficulty	Build	Score	ERC/DRC	Dominant observed failure
mcp23017_breakout	Very easy	1	0.10	No	Missing functional circuitry and residual shorts
arduino_nano_board	Easy	1	0.05	No	Wrong connector mapping and unpowered active device
i2s_audio_phat	Medium	1	0.00	No	Power-ground and power-signal copper shorts
asac_fc_rev_b	Hard	1	0.10	No	Incomplete realization, split connections, and a short
adau1452_dsp_core	Very hard	1	0.00	No	Obstacle-unaware routing and extensive copper shorts
cm4_baseboard	Extreme	1	0.00	No	Missing active realization, invented mappings, and shorts

A.1 Retained-trace workflow and source-use details

The first three traced tasks use neither native web search nor a subagent. In the ASAC task, the root agent downloads and parses an exact pinned upstream XML netlist; a spawned research agent also leaves a scratch clone, although the child’s commands are not present in the retained root stream. In the ADAU1452 task, the agent downloads the exact upstream PCB and three upstream KiCad schematics into scratch, then emits a smaller byte-distinct derived project. A final-file hash or route-overlap test would miss that source use. In the CM4 task, the observed trace uses public board pages, pinout information, and the official CM4 datasheet but does not download or copy the upstream KiCad project. These are public-source reconstructions, not closed-book runs.

Artifact inspection explains the low scores. The MCP board realizes its main interface but omits much of the reference’s passive, filtering, and protection circuitry and retains signal shorts. The Arduino output reconstructs a Mini-B-style Nano variant rather than the task’s connector mapping, supplies a label-only schematic, and leaves an active device unpowered. The I2S board contains severe power-ground and power-signal copper shorts. ASAC creates the named active blocks but retains split and isolated external connections, incomplete component-function coverage, and a residual short. ADAU1452 and CM4 use obstacle-unaware direct-segment routing without vias, planes, or a physical verification pass, producing large numbers of cross-net intersections; both also submit text-only schematics. The CM4 run additionally assigns unused M.2 lane positions to camera signals even though the CM4 exposes only one native PCIe lane.

B Harness settings, prompts, traces, and agent interventions

B.1 Reasoning-setting and prompt controls

Table 8 states the exact comparison supported by each harness. The Pi registry contains no non-high configuration or result. Although both runners use the string `high`, they pass it through different interfaces and providers; the archive does not establish equal reasoning tokens, compute, latency, or cost.

The exhaustive local prompt inventory contains 642 evaluated prompt artifacts from 37 campaigns over the original 40 tasks: 554 Codex prompts and 88 Pi prompts. All 642 explicit system-prompt files are empty. Each task body has one hash across every campaign. There are exactly two wrappers: Codex adds a subagent instruction and a goal-persistence instruction; every other benchmark-supplied line is shared.

The runs therefore hold prompt text fixed within the reported setting and Pi-model comparisons, but do not estimate the effect of changing a prompt. A causal prompt experiment must hold model, task, tools, access, image, sampling, and seed policy fixed while varying one instruction.

Table 8: Archived reasoning-setting coverage. The Codex rows are the only within-model setting sweep.

Model and harness	Settings	Tasks per setting	Runner control and supported inference
GPT-5.5 through Codex	low, medium, high, xhigh	35	Codex effort flag; descriptive within-model comparison
Gemini 3.1 Pro Preview through Pi	high only	35	-thinking high; one complete-system baseline
DeepSeek V4 Pro through Pi	high only	35	-thinking high; one complete-system baseline

Table 9: Prompt identity over the 35 released tasks. “Full” includes the harness wrapper.

Comparison	Same task body	Same full prompt	Interpretation
Four Codex reasoning settings	35/35	35/35	Setting differences are not caused by benchmark prompt changes
Pi Gemini versus Pi DeepSeek	35/35	35/35	Model/provider systems receive identical benchmark prompts
Codex versus Pi	35/35	0/35	Wrapper, model, provider, CLI, and tools change together

B.2 Trace scope and preliminary same-task case

Most failed Table 6 rows do not contain evidence of a completed model workflow. Codex low has four immediate authentication failures, Codex high has 24, and Codex xhigh has 14 runtime or model-stream failures. For Pi, 31/35 Gemini rows and all 35 DeepSeek rows record zero tokens, an empty response, and no project; three other Gemini rows use tokens but produce no project. We count these as complete-harness outcomes, not as evidence that a model chose a particular tool sequence or engineering stopping point.

Two preliminary USB-C runs retain complete traces but are not part of Table 6 or the predeclared six-task study. Table 10 gives their observable contrast. The Gemini run explicitly downloads and processes the exact upstream schematic and PCB despite the no-copy task instruction. This establishes source use in that preliminary run, not a general Gemini–Codex difference or a claim about the final Gemini ADAU1452 run, whose trace is absent.

Table 10: Preliminary full traces on the same USB-C task. These one-task runs are provenance examples, not model-level estimates.

Harness	Observable actions	Source use	ERC/DRC	Score
Codex low	26 commands; one nonzero exit	No web or shell fetch	No	0.05
Pi/Gemini high	84 tool calls; 13 tool errors; two successful board loads	Five shell downloads, including exact upstream Ki-Cad files	No	1.00

B.3 Board-level failures and intervention priorities

Table 11 counts each strict submitted-grader mechanism at most once per saved project. Rows overlap. DeepSeek has no saved project and is excluded from the denominator.

Weak and incomplete variants lead to the same repair action. Under that broader grouping, 92/99 projects have external-mapping problems, 86/99 have required-circuit-realization problems, 73/99 have isolated or split connectivity, 70/99 contain shorts, 33/99 have active-power problems, and 12/99 have high-speed-routing problems. At least one of mapping, circuit realization, connectivity, or shorts appears in 97/99 saved projects.

Table 11: Strict board-level failure counts over the 99 archived saved projects. Column denominators are 31, 35, 11, 21, and 1.

Failure mechanism	Low	Med.	High	Xhigh	Gemini	Total
Isolated or split external net	24	28	8	12	1	73
Wrong external pin mapping	27	29	8	12	0	76
Signal or power short	30	27	4	9	0	70
Missing required active path	28	25	9	8	0	70
Unpowered active device	8	6	2	1	1	18
Missing or implausible high-speed routing	2	4	1	5	0	12

The resulting intervention sequence is: (1) validate a typed connector-to-net plan; (2) track every required circuit block and its power, ground, enable, and reset paths; (3) provide ERC, DRC, connectivity, and short checks; (4) feed exact failures back through a mandatory repair loop; (5) validate every source import or geometric transformation; and (6) run task-specific simulation after structural checks pass. The current evidence motivates this order but does not measure its causal benefit. A clean study should add these stages one at a time while holding the model, task, setting, tools, access, image, sampling, and seed policy fixed, and report score, failure frequencies, repair rounds, time, tokens, and cost.

C Extended related work

Execution-based software-agent benchmarks. Execution-based software benchmarks evaluate agents by running the artifacts they produce. SWE-bench frames software engineering as repository-level issue resolution, where an agent edits a real codebase and must pass tests associated with the target issue [Jimenez et al., 2023]. OSWorld evaluates multimodal agents in real desktop and web environments with custom execution-based evaluation scripts [Xie et al., 2024]. ProgramBench studies a reconstructive setting in which agents rebuild software from an executable and documentation, and behavioral tests compare the candidate program with the reference executable [Yang et al., 2026]. EDA Bench applies this behavioral-evaluation perspective to PCB reconstruction, where correctness depends on realized geometry, external connectivity, and electrical-interface behavior rather than software tests alone.

CAD generation and geometric evaluation. Text2CAD introduced a large-scale text-to-parametric-CAD dataset and model for generating CAD command sequences from natural-language prompts [Khan et al., 2024]. CadEval and CAD Arena provide evaluation settings for text-to-CAD models using geometric validity, rendering, and prompt-level comparisons [Epoch AI, 2026, CAD Arena, 2026]. CADSmith emphasizes programmatic geometric validation for generated CadQuery models [Barkley et al., 2026]. These works motivate execution-based evaluation for engineering artifacts, but their primary correctness signals are geometric rather than electrical. PCB reconstruction also requires connectivity, short avoidance, active-circuit realization, and external-I/O behavior.

Circuit, schematic, and PCB benchmarks. CircuitLM and PCBSchemaGen generate or synthesize circuit schematics from natural-language prompts and use structural or constraint-based checks [Al Hasan et al., 2026, Zou et al., 2026]. SchGen studies PCB schematic generation using semantic-grounded code representations [Luo et al., 2026]. OmniSch evaluates multimodal schematic understanding and schematic-to-graph reasoning [Lu et al., 2026]. HWE-Bench evaluates board-level schematic design with datasheet context and hardware bug repair [Qiu et al., 2026, Cui et al., 2026]. PCB-Bench evaluates LLM reasoning over placement, routing, and PCB design comprehension [Li et al., 2026]. EDA Bench differs by requiring a complete routed KiCad PCB project and scoring the realized board through external-I/O simulation and PCB-geometry checks.

D Extended scope and limitations

The 35-task design gives EDA Bench broad skill coverage while keeping a complete tool-using sweep practical. This is the same scale regime used by expensive open-ended benchmarks: MLAGentBench uses 13 experimentation tasks, RE-Bench uses seven research-engineering environments, PaperBench uses 20 full-paper replications, and PostTrainBench applies one open-ended post-training task

formulation across four base models and seven target benchmarks [Huang et al., 2024, Wijk et al., 2024, Starace et al., 2025, Rank et al., 2026]. Curated small subsets can also preserve useful signals from larger suites. TinyBenchmarks reports that 100 selected examples can estimate the 14,000-example MMLU benchmark, while SWE-bench Verified Mini reports that 50 of 500 tasks approximately preserve performance, test-pass-rate, and difficulty distributions across 16 models while reducing storage from 130 GB to 5 GB [Polo et al., 2024, Hobbhahn, 2025].

Within its stated population of public, source-backed KiCad projects with externally testable behavior, EDA Bench covers different board families, functional skills, and difficulty levels. Each task contributes a source design, frozen reference, expected-I/O files, controlled faults, license review, and grader support. The resulting size makes complete six-harness sweeps, an all-task fault audit, alternative weighting, and task-bootstrap analysis practical. Additional source-backed packs and a sealed private split can extend coverage to proprietary and currently omitted workflows.

The grader uses deterministic generic behavioral models and does not prove electrical safety, regulatory compliance, high-speed protocol compliance, thermal reliability, power sequencing, or production readiness. Richer per-task SPICE, IBIS, S-parameter, and field-solver models would improve fidelity. The revised single-fault sweep contains 19 genuine unchanged misses after manual requirement-level review and, separately, 58 faults whose scores decrease but remain at least 0.95. The unchanged misses are concentrated in complete route-removal and active-power edits, while threshold calibration is especially permissive for local passive and single-pad edits.

The rigid-transform controls all score 1.0, but they preserve reference connectivity, relative placement, and routing and therefore do not test an independently designed circuit or independently routed PCB. Other representation controls show false rejections: a plausible uniform route-width edit scores 0.25, and generic signal-net names score 0.65 despite unchanged topology. The evidence supports invariance to the tested rigid coordinate transform, not general calibration across all electrically equivalent implementations.

We ran one fresh complete medium-reasoning sweep, but one additional sample does not estimate stochastic variance; task-bootstrap intervals do not replace repeated runs. We also have not measured agreement between the grader and blinded PCB engineers. The 99 archived event files are omission records, so those rows support artifact-level failure analysis but not reconstruction of tool sequence or browsing history. The six-task trace sample supports limited root-agent workflow observations, but it is a one-run-per-difficulty case study using a separately pinned pre-Harbor environment, and one spawned child stream was not retained.

E Validation controls

The broader revised-grader sweep in Table 5 applies swapped-pin, route, short, active-power, component-removal, and high-speed edits across every applicable released task. It keeps both high-scoring false negatives and low-scoring benign controls. The rigid-transform suite adds a positive representation test: eight complete reference PCB layouts were rotated by 180 degrees and translated, and all eight score 1.0 with no caps. Two transformed boards omit only the non-electrical FD1 fiducial because KiCad 10 reserialization otherwise introduces a self-overlap DRC error. These are electrically identical, reference-derived layouts, not independently designed circuits or independently routed boards.

E.1 Fault response and oracle-component ablation

For each row in Table 14, we remove that mechanism’s weighted subscores, renormalize the remaining positive weights, remove only its assigned caps, and hold all lower-level diagnostics fixed. The 35 references and 295 controlled faults use the later DRC-rule-fixed revised-grader audit; the 99 saved projects use their frozen revised-grader diagnostics. Because the final score is the minimum over interacting caps, the leave-one-out effects do not add to the total score.

E.2 Historical upstream versions

As a second check, we graded 13 distinct board directories preserved in exact historical commits or versioned releases of seven pinned upstream projects. These are not synthetic mutations and

Table 12: Reviewer-facing validation controls for score interpretation.

Control	Observed result
Submitted-grader reference project	35/35 released references score 1.0
Submitted-grader structural fail canary	35/35 released fail canaries score at most 0.15, below the 0.75 release threshold
Generated mutation canaries	162 generated mutation canaries across the original 40-task candidate set pass validation
Calibrated task-family checks	USB-C, RS-485, BQ24295, and M.2/PCIe references score 1.0 and their mutation canaries remain below threshold
Revised-grader references	Mean 0.957; 33/35 at least 0.95 and two at 0.25
Revised-grader internal-refdes variants	Mean 0.855; 24/35 at least 0.95
Rigidly transformed PCB layouts	8/8 electrically identical 180-degree-rotated and translated layouts score 1.0 with no caps
External connector refdes rename	USB-C reference scores 0.992 rather than 1.0
Missing routed copper	USB-C and NanoUPDI route-removal controls score 0.0
Plausible route-width edit	USB-C route-width edit scores 0.25, an adverse representation-sensitivity result
Generic signal-net names	USB-C topology with six generic signal names scores 0.65
Implausibly tiny traces	Manufacturability cap applies and USB-C tiny-trace control scores at most 0.35
Missing or unparsable project	Returns 0.0 and build failure

Table 13: Score response for all 295 applicable revised-grader fault rows. “High” means a final fault score of at least 0.95. A high but decreased row is a threshold-calibration miss, not a completely unchanged score.

Intervention	N	Fell	Same	High	High/fell	High/same
Swap connector pins	35	35	0	3	3	0
Disconnect external pad	35	33	2	9	9	0
Remove external route	34	27	7	7	0	7
Tie signal to ground	35	35	0	5	5	0
Disconnect active power pad	31	25	6	8	4	4
Remove all active power/ground	31	29	2	5	5	0
Remove signal or decoupling passive	34	33	1	23	23	0
Remove active device	32	32	0	9	9	0
Remove high-speed route	28	22	6	6	0	6
Total	295	271	24	75	58	17

are byte-distinct from the frozen task reference except where noted. Scores range from 0.050 to 1.000: five versions score 1.000, one scores 0.997, one scores 0.550, one scores 0.050, and five score 0.200. For example, two older USB-C breakout releases score 1.000, while a camera-adapter version immediately preceding an upstream I2C-line swap scores 0.200. Low scores can reflect a real product change rather than grader error, so this check supports sensitivity to released design changes but does not replace independently authored equivalence or engineer correlation.

Table 14: Leave-one-mechanism-out counterfactual recomputation under the revised grader. Each delta is ablated minus baseline; a negative separation delta means the mean intact–fault gap shrinks. Baseline means are 0.957143 for references, 0.354567 for faults, and 0.012121 for the 99 saved projects; the baseline paired gap is 0.599671.

Mechanism removed	Δ reference	Δ fault	Δ separation	Δ saved
External-interface contract	0.000000	+0.014959	-0.014959	+0.012626
Connectivity and short safety	0.000000	+0.016516	-0.016516	+0.007576
Internal-circuit realization	0.000000	+0.018113	-0.018113	+0.000505
KiCad ERC/DRC rules	+0.042857	+0.447667	-0.401904	+0.032828
Fabrication and impedance	0.000000	-0.000045	+0.000045	0.000000
Simulation response	0.000000	-0.000086	+0.000086	0.000000
High-speed geometry	0.000000	+0.004370	-0.004370	0.000000

Table 15: Historical-version examples from pinned upstream repositories.

Task	Historical version	Score
CM4 CSI adapter	immediately before upstream I2C-line swap	0.200
FTDI toolkit	before final DRC/ERC fixes	0.997
FTDI toolkit	earlier layout revision	0.050
NanoUPDI	v2.2 layout before fabrication cleanup	1.000
NanoUPDI	earlier v2 layout	0.550
OV9281 dual camera	immediately before lens-DNP fix	1.000
Pi485	v1.1.1 standalone board	0.200
Pi485	earlier high-side-switching board	0.200
USB-C breakout	v1.0.0 / v1.1.1	1.000 / 1.000
USB-UART CH340C	v1.0.0 / v1.1.1	0.200 / 1.000

F Failure-mode diagnostics

Table 16: Common functional failure modes exposed by the grader. These failures are typical of buildable submissions and illustrate why KiCad parseability is not sufficient for functional PCB reconstruction.

Failure mode	Observable symptom	Why it affects score
Split required external nets	Pins that should belong to the same external interface are assigned to disconnected copper regions or separate nets	Boundary stimuli cannot propagate through the declared interface, even when connector footprints are present
Isolated external I/O pads	Required pads are placed but not connected to routed copper or active circuitry	The board exposes a physical pad without realizing the electrical behavior required by the task
Wrong pin or net mapping	External connector pins are swapped, mislabeled, or attached to semantically incorrect nets	The board may appear visually plausible while driving or observing the wrong external signal
Missing active realization	Components required for level shifting, power-path behavior, transceiver behavior, or control logic are absent or not connected	Passive connectivity alone cannot implement the task-required circuit behavior
Weak active-device power integrity	Active devices are present but lack required power, ground, or enable connectivity	The simulated or inferred behavior cannot be attributed to a realizable powered circuit
Implausible high-speed geometry	Differential or high-speed-like interface families are missing, weakly routed, or routed with implausible geometry	Dense carrier and compute tasks require not only pin labels but routed interface families with plausible physical realization

G Baseline provenance

The May 6, 2026 submitted result table and the revised-grader comparison use the same saved final projects without rerunning model harnesses. The revised audit includes 210/210 task/configuration records, verifies all 341 archived project files against the pinned provenance revision, and keeps the four documented May 6 task-level overrides separate from raw grader output. Source runs were: `codex__gpt-5.5-web-low_20260505T200208Z_2719115`, `codex__gpt-5.5-web-medium_20260506T010715Z_2812175`, `codex__gpt-5.5-web-medium_20260506T010715Z_2812182`, `codex__gpt-5.5-web-medium_20260506T032737Z_2884928`, `codex__gpt-5.5-web-medium_20260506T010716Z_2812190`, `codex__gpt-5.5-web-high_20260505T200208Z_2719113`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884940`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884938`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884943`, `codex__gpt-5.5-web-xhigh_20260506T034126Z_2891808`, `pi__gemini-3.1-pro-preview-web-high_20260505T184850Z_2697190`, and `pi__deepseek__deepseek-v4-pro-web-high_20260505T200208Z_2719116`.

The executable regrade entry point is `uv run regrade-provenance`. For example, the Codex medium row is reproduced by passing its four source runs:

```
uv run regrade-provenance \
  --run codex__gpt-5.5-web-medium_20260506T010715Z_2812175 \
  --run codex__gpt-5.5-web-medium_20260506T010715Z_2812182 \
  --run codex__gpt-5.5-web-medium_20260506T032737Z_2884928 \
  --run codex__gpt-5.5-web-medium_20260506T010716Z_2812190
```

By default the command uses the current released task manifest, downloads saved submission/final_project directories from Hugging Face provenance, and runs the current local grader.

Completed no-web calibration runs are present under these provenance paths: `evals/harnesses/codex__gpt-5.5-no_web-medium_20260507T033319Z_3432012`,

evals/harnesses/codex__gpt-5.5-no_web-high_20260507T033319Z_3432018,
evals/harnesses/codex__gpt-5.5-no_web-xhigh_20260507T033319Z_3432017,
evals/harnesses/codex__gpt-5.4-no_web-xhigh_20260507T033528Z_3439753, evals/
harnesses/codex__gpt-5.4-mini-no_web-medium_20260507T033724Z_3448333, and
evals/harnesses/codex__gpt-5.4-mini-no_web-high_20260507T033724Z_3448341.

H Cost and compute accounting

The completed GPT-5.5 web runs used storage-light provenance. The combined Codex GPT-5.5 low/medium/high/xhigh campaign uploaded about 185.6 MiB of Hugging Face provenance. Across all six final harnesses, the total uploaded provenance was about 193.1 MiB.

Table 17: Recorded Codex GPT-5.5 web campaign accounting.

Harness	Estimated OpenAI API USD	Estimated Codex credits	HF provenance
Codex low	38.22	955.41	48.9 MB
Codex medium	84.40	2110.05	49.5 MB
Codex high	49.39	1234.64	27.6 MB
Codex xhigh	182.75	4568.64	68.6 MB

The estimates are computed from recorded token usage, current provider pricing, and the Codex token-based rate card. They are included to make leaderboard submissions auditable and to discourage hidden cost and provenance blowups.

I Release and license gate

EDA Bench separates runtime prompts from full task packs with references, snapshots, controlled faults, and expected-I/O files. For NeurIPS release, full packs with unlicensed, upstream-terms-only, noncommercial, or otherwise restricted source snapshots must be handled by one of three policies: obtain explicit redistribution permission, remove the source snapshot and provide a reproducible acquisition script, or quarantine the task from the public full pack while retaining runtime metadata.

The current public release quarantines five candidate tasks under this conservative rule: `cm5io_official`, `mixed_signal_stm32_dev_board`, `rp2350b_dev_board`, `stm32f_audio_codec`, and `ef28_badge`. The public score-supported full-pack release has 35 tasks. These task ids may remain documented as candidates, but their references, source snapshots, and canaries are not part of the released full-pack artifact.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the paper’s scope as introducing an execution-based benchmark for web-enabled PCB construction agents, documenting its harness protocol and external-I/O evaluator, validating the scoring artifact with canaries, and reporting initial harness baselines. The paper explicitly excludes hidden-answer circuit invention, fabrication readiness, regulatory certification, and vendor-accurate electrical validation.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.

- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The Limitations section discusses public-source task exposure, task-set size, evaluator fidelity, and upstream license constraints. The benchmark overview and access-policy calibration sections also clarify that the default baselines are web-enabled and should not be interpreted as closed-book PCB invention.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs.
- The authors should reflect on the factors that influence the performance of the approach.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that are not acknowledged in the paper.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is a benchmark and empirical evaluation paper and does not include theoretical results or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper reports the task-pack revision, harness protocol, scoring formula, runtime and grading artifact split, run-record dataset, source run identifiers, and regrade procedure. The code, released task artifacts, grading artifacts, and saved final projects are public or described through the release and quarantine policy.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Appendix A identifies the source repository, Hugging Face task dataset, task-pack revision, Croissant metadata, and public run-record dataset. Appendix I describes the license triage policy and states that five upstream-license-risk candidate tasks are quarantined from the released full-pack artifact.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer.
- The instructions should contain the exact command and environment needed to run to reproduce the results.
- The authors should provide instructions on data access and preparation.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines.
- At submission time, to preserve anonymity, the authors should release anonymized versions, if applicable.
- Providing as much information as possible in supplemental material is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The benchmark overview, harness protocol, scoring evaluator, artifact validation, and experiments sections describe the task interface, Dockerized tool environment, model and harness configurations, score definition, build-success definition, task count, validation controls, and access-policy separation.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper states that the reported results are complete released-task sweeps rather than repeated independent sampling campaigns. It therefore does not claim statistically significant pairwise model rankings. The supported conclusions are artifact-level and qualitative: the benchmark is not saturated by current evaluated harnesses, build success diverges from functional score, and access policy affects the evaluated harnesses under the reported protocol.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated.
- The method for calculating the error bars should be explained.
- The assumptions made should be given.
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- For asymmetric distributions, the authors should be careful not to show symmetric error bars that would yield results that are out of range.
- If error bars are reported in tables or plots, the authors should explain how they were calculated and reference the corresponding figures or tables.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper specifies the Dockerized KiCad, ngspice, Python, and Node environment, records run identifiers and run records, and reports cost and stored artifact volume for the main Codex campaign in Appendix H. Saved run records include additional execution details for reproduced runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work is a benchmark and evaluation study without human subjects or deception. The paper discusses limitations, release constraints, and the risk of overtrust in generated hardware artifacts.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper frames the benchmark as a tool for measuring EDA-agent reliability and explicitly states that scores are not evidence of manufacturing readiness, safety, EMC compliance, regulatory compliance, or production readiness. The main negative impact is overtrust in generated hardware artifacts. The mitigation is conservative scoring, validation controls, and explicit scope limits.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses, fairness considerations, privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, when it is being used as intended but gives incorrect results, and from misuse.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases benchmark tasks, grading assets, and evaluation code rather than a new high-risk generative model or scraped personal or sensitive dataset. Upstream-license-risk hardware snapshots are quarantined from the released full-pack artifact until permission or acquisition-script replacement is available.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model.
- Datasets that have been scraped from the Internet could pose safety risks.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Appendix I states the license triage policy and identifies the five quarantined candidate tasks. The repository includes LICENSES.md, task-pack license notes, and source-attribution metadata.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license should be included for each asset.

- For scraped data from a particular source, the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The paper documents the task taxonomy, runtime and full-pack split, scoring evaluator, validation controls, run records structure, Croissant metadata, artifact availability, and release policy. The repository and dataset contain detailed task and release documentation.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets if applicable.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[N/A\]](#)

Justification: The paper does not involve human-subjects research.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval or equivalent may be required for any human subjects research.

- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity, if applicable, such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not important, original, or non-standard components of the core method development in this research.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.